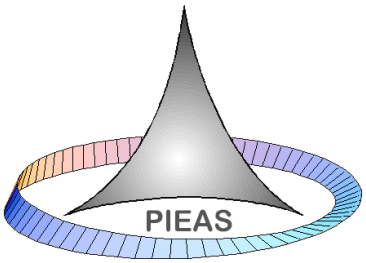
Development of an Anomaly Detection System for a Nuclear Power Plant using Transformer Models

By

Muhammad Daniyal Nazeer

Thesis submitted to the Faculty of Sciences at PIEAS in partial fulfillment of requirements for the Degree of BS Physics

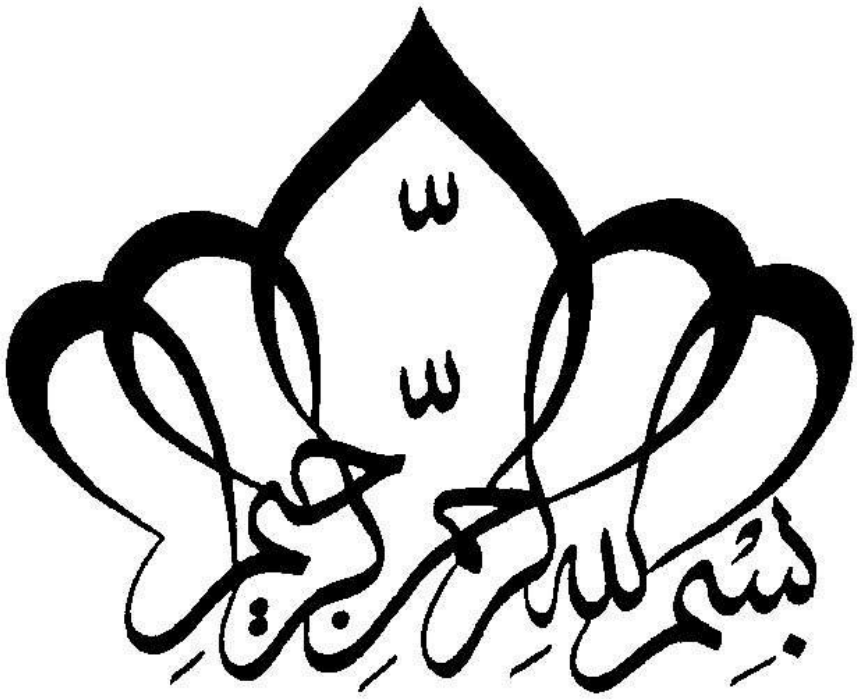


Department of Physics and Applied Mathematics

Pakistan Institute of Engineering & Applied Sciences

Nilore, Islamabad, Pakistan

July, 2025


بسم الله الرحمن الرحيم

Department of Physics and Applied Mathematics,
Pakistan Institute of Engineering and Applied Sciences (PIEAS)
Nilore. Islamabad 45650, Pakistan

Declaration of Originality

I hereby declare that the work contained in this thesis and the intellectual content of this thesis are the product of my own work. This thesis has not been previously published in any form nor does it contain any verbatim of the published resources which could be treated as an infringement of the international copyright law.

I also declare that I do understand the terms ‘copyright’ and ‘plagiarism’, and that in case of any copyright violation or plagiarism found in this work, I will be held fully responsible for the consequences of any such violation.

Signature: _____

Name: _____

Date: _____

Place: _____

Certificate of Approval

This is to certify that the work contained in this thesis entitled

Development of an Anomaly Detection System for a Nuclear Power Plant using Transformer Models

was carried out by

Muhammad Daniyal Nazeer

Under our supervision and that in our opinion, it is fully adequate, in scope and quality, for the degree of BS Physics from Pakistan Institute of Engineering and Applied Sciences (PIEAS).

Approved By:

Signature: _____

Supervisor: ***Dr. Asifullah Khan, DCIS***

Signature: _____

Co-Supervisor: ***Dr. Awais Zahur, DNE***

Verified By:

Signature: _____

Head, Department of Physics and Applied Mathematics

Stamp:

Dedication

With deep gratitude, I dedicate this project to my family, especially my father. Their love, support, and prayers have been with me every step of the way. In times when I needed them the most, they were my strength—offering comfort, understanding, and motivation.

I am truly thankful for their patience and unwavering faith in me. Their presence reminded me how important it is to have people who genuinely believe in you. This journey wouldn't have been the same without them, and I share this achievement with all of them from the bottom of my heart.

Acknowledgement

Among the first I would like to thank ALLAH – our Creator, our Guide, our Helper, and the One who blesses us with everything we have. He forgives our mistakes and continues to show mercy, even when we fall short. Without His endless blessings, none of this would have been possible.

I would pay thanks to Dr. Asifullah Khan as well Dr. Awais Zahur, my respected supervisor and co-supervisor respectively, for help, corrections, and feedback during the project. Their knowledge, encouragement, and mentorship played a big part in helping me complete this work and stay on the right track.

I am thankful to my family for always being there for me — for their support, patience, and understanding during this journey. Lastly, I want to give special thanks to my parents. They have always been my biggest support, standing by me through every challenge and helping me reach this milestone. Their love and sacrifices mean everything to me.

Muhammad Daniyal Nazeer

Contents

List of Figures

Abstract

Ways of ensuring safety and reliability of nuclear plants (NPPs) are important because of their significant role in production of energy and also potential threats associated with their operations. The following thesis highlights ongoing procedures in the creation of an anomaly detection system for NPPs through the capabilities of transformer-based machine learning model. Traditional anomaly detection system focuses more on conventional statistical approaches, and this approach may struggle with high-dimensional data. But in the following project, a transformer framework is designed and implemented to address these challenges because of its reliable performance in natural language processing and sequence analysis tasks. The proposed architecture was trained on simulated dataset called Nuclear Power Plant Accident Dataset (NPPAD) generated by PCTRAN (Personal Computer Transient Analyzer) for 18 different nuclear accidents and normal behavior operation. The reactor was two loop Pressurized Water Reactor (PWRs). Before the sequential analysis over time, classification of accident classes was done using varied neural architectures as well. The main goal is to detect any deviations from the very normal behavior and therefore to report any anomalies in real time. The research methodology involves extracting data from Nuclear Power Plant Accident Data (NPPAD) repository and transforming it into a suitable format through data preprocessing to ensure reliability for machine learning. The processed data was set into training and testing sets. Finally, a model architecture was designed, and hyperparameters were tuned to achieve optimal performance. Transformers are best known for effectively handling dependencies and patterns in data in parallel, thereby outperforming any other traditional AI Models such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

Chapter 1: Introduction

Nuclear Power plants (NPP) requires excellent monitoring systems to ensure safe operations and early detection of any harm to any equipment of operation, i.e., detection of potential anomalies. Following project explores the applications of transformer model, which essentially made significant improvements in the areas of natural language processing (NLP) [1] and large language models (LLMs) [2], to create a better reasonable anomaly detection system for NPPs. The starting phase of the project was concentrated on three key areas: first the understanding of transformer architecture and NPP design with its working, second was to study different NPP accidents with their cause, effect and mitigation, and the final phase was the searching and identifying appropriate datasets, and coding data processing pipelines. Main phase of the project focused on building varied neural architectures, their training and implementation. In all those types of nuclear power plants, the project focuses mainly on pressurized water reactors (PWR) because these are the predominant types of power reactors which are categorized under the category of light water reactors (LWRs). These LWRs constitutes nearly 80% of world's reactors and more than three-quarters of LWRs are PWRs [3].

1.1 Background

The nuclear power had adopted the first generation of reactors that were planned, built and operated since the 50s of the past century to demonstrate the electric power. These power plants were replaced by the second-generation reactors also known as Gen-II reactors. These Gen-II reactors which mainly have been commissioned during the 1970s and 1980s have taken up the larger part of the available nuclear power on earth. A Pressurized Water Reactor (PWR) is such type of reactor (Gen-II reactor) which is commonly used in power plants to generate electricity. It uses water as both coolant and moderator, and importantly its design makes sure that water in the reactor core remains in a liquid state by keeping it under high pressure. The first pressurized water nuclear reactor was developed by Westinghouse Electric Company that used the technology initially constructed in the realization of the U.S. nuclear submarine program [4].

1.2 Motivation

Nuclear energy is a cornerstone of global hard works to meet the required growing energy demands with very reduced carbon emissions. The safety of operations and obviously reliability of nuclear power plants are critical to avoid incidents that could have widespread environmental and society related consequences. As the real-time data produced by the modern NPPs creates a dire need for more advanced and adaptive solutions to monitor, detect and handle anomalies with much efficiency. Advances in machine learning have demonstrated AI capabilities to process huge amount of data and find patterns. Transformer models offers edge solutions for excellent anomaly detection. Through efficient anomaly detection system, we can enhance safety, minimize operational downtime and also reduce maintenance costs. By preventing any hazards before occurrence, such systems can contribute to public trust in nuclear energy as reliable power source [5].

1.3 Objectives

The primary purpose of the said project goal was to conceptualize and develop an anomaly detector according to a transformer-based system in NPPs to enhance their security in operative terms. Specific goals of the study included building a deeplearning framework capable of handling high-dimensional and time-series data generated by NPPs but in this case it was PCTRAN simulated data. After utilizing capabilities of transformer models to capture dependencies and patterns we aim for achieving high precision and recall in detection thereby reducing false positives and false negatives. Finally, demonstrating potential of advanced AI techniques in improving safety and trustworthiness of crucial energy infrastructure [6].

1.4 Scope

As the study was specifically being done for NPPs, simulated datasets of NPP operations were used to train and evaluate the system, therefore to make the findings have relevance to attempt to equate to the actual world circumstances. Study did not include its involvement in actual NPP but focuses more on simulated environments. Furthermore, the implementation was restricted to transformers only for sequential forecasting part but depending upon the need for comparison other architectures such as Multi-layer Perceptrons (MLPs), Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) for classification part were included as well.

1.5 Thesis Organization

This thesis will have 04 chapters. The first chapter introduces the concept of PWRs while its related accidents are described in chapter 02. The third chapter focuses on working of transformers architecture and other neural frameworks used for classification part that are MLPs, CNNs and RNNs and how they handle data. Fourth chapter elaborates data preparation and preprocessing with implementation of algorithms and finally about results including loss curve convergences and comparison of different architectures.

Chapter 2: Literature Review

2. Pressurized Water Reactors

The moderator and coolant functions which are fulfilled by light water within the core of the reactor in a PWR. Energy is generated through fission reaction of atoms (fissile in nature) that is contained in the fuel, which is utilized to heat the primary loop water. Under very high pressure that is around 155 bar water stays in the liquid form. The heated water is then directed into the steam generator, this is whereby it gives out its thermal energy to the secondary loop water. The water in the second loop is changed into steam. The resultant steam then acts as driver for the turbines linked to generators to produce electricity [4].

2.1 Components of a PWR

Because nuclear power plant is a complex design with hundreds of valves and pumps, vast amounts of twisting of tubing, electric wiring and structural steel are necessary. Nevertheless, to speak about them, several components are important here. Those are pressurizer, steam generators, main circulation pumps, pressure vessel, generator, heater, condenser and containment structure.

2.1.1 The Pressure Vessel

Figure 2-1 shows a common PWR pressure vessel. Its length is about 13m with a diameter of 5m. Low alloy steel is used in its structure with a thickness of 20cm. This wall has to be such that it bears the pressure of 150bar. The primary cycle water enters this with the inlet nozzles and this water flows downward between the vessel and the core barrel and flows up the reactor core, when it drains the heat of the needed fuel pins and passes out of the vessel by way of the outlet nozzles [3] [4].

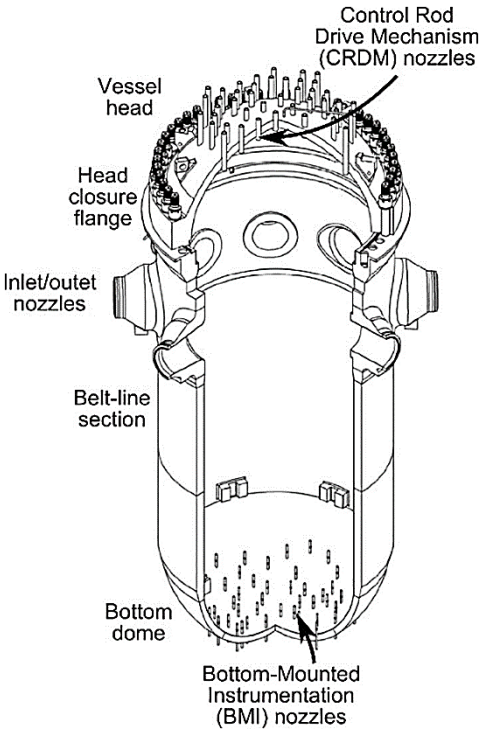


Figure 2-1: Pressure Vessel [3]

2.1.2 Recirculation Pumps

The recirculation pumps control the flow of the reactor core in primary loop. These pumps are vertical single stage centrifugal types of pump that can serve virtually the entire life of the plant that is nearly over 30 years. It is made of stainless-steel. Such pumps cause a continuous, and highly accurate controlled flow of coolant to sustain pressure [4].

2.1.3 The Pressurizer

PWR main system is the constant volume system. When there is an increase or decrease in the temperature of the primary water, the water would either expand or instead contract. Water is incompressible almost, so moderate changes of temperature lead to large changes of pressure. The basic loop has a pressure-control surge vessel known as pressurizer, as shown in the Figure 2-2, to avoid the pressure surges. It is a cylindrical tank and there is water in the bottom of the tank and steam in the top. The steam is compressible and this means that it absorbs the abrupt changes in

pressure. When the water temperature drops, the water in the pressurizer contracts, which leads to a drop-in pressure. Some water is converted into steam, restoring pressure due to the actuation of heaters at base. If temperature rises, water expands into the pressurizer, raising steam pressure. Valves then inject spray water into the top to condense steam and reduce pressure back to normal [4].

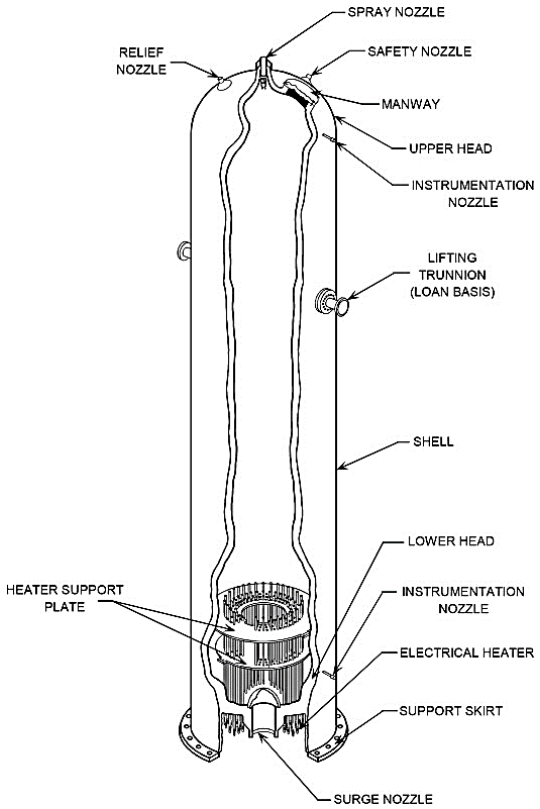


Figure 2-1: Pressurizer [30]

2.1.4 Steam Generators

The steam generators, displayed in Figure 2-3 in PWR, work as the heat transferring domains, in which the heat energy of the main water in primary is transferred to the secondary one to produce the steam in the downstream turbo generators. These generators are mostly bigger in comparison

with pressure vessels. Said generators have an inner mechanism that causes high pressure water delivered by a primary source to travel through a series of pressure tubes of the U-form. A part of the heat energy of primary water is transferred to secondary water at an approximate pressure of 1000psi along the periphery of pressure tubes and thereafter secondary water conveys into steam. The steam can be having droplets of water above the tubes. In it, there are a number of cyclone separator within which these droplets are eliminated and only drier steam enters the high-pressure turbine. This is where the primary water exits generator with a temperature of ~ 300 degrees C and is returned into the reactor [4] [7].

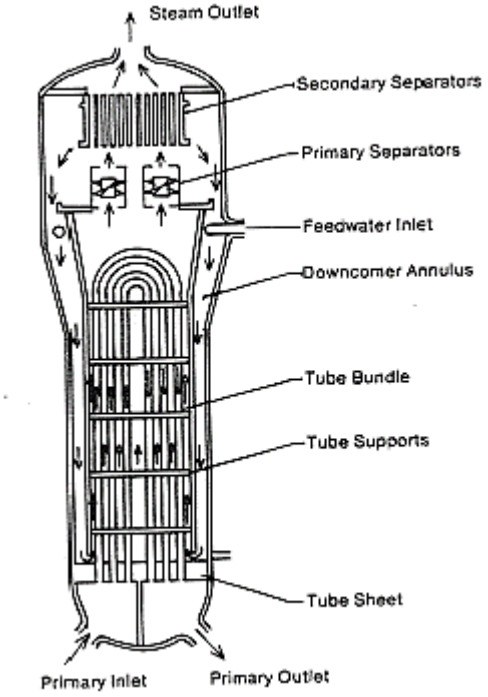


Figure 2-2: Stream Generator [3]

2.1.5 The Containment Building

In safe use of nuclear power, it is highly important to separate the radioactive fission products with the environment of biosphere. In order to prevent this leaking of the radioactive substances out of the plant, there are three main barriers of isolation applied in all nuclear power plants. The first is

fuel cladding, that seals most of the fission fragments out in the primary coolant but some of them might escape. The second degree of confinement is offered through the isolated primary loop and the pressure vessel. Finally, just in case there is a possibility that there can be a leak in the primary system, the reactor, and the critical features along which the primary water flows are sealed inside the containment building, which is shown in Figure 2-4, which is well built to resist tornados and other natural disasters.

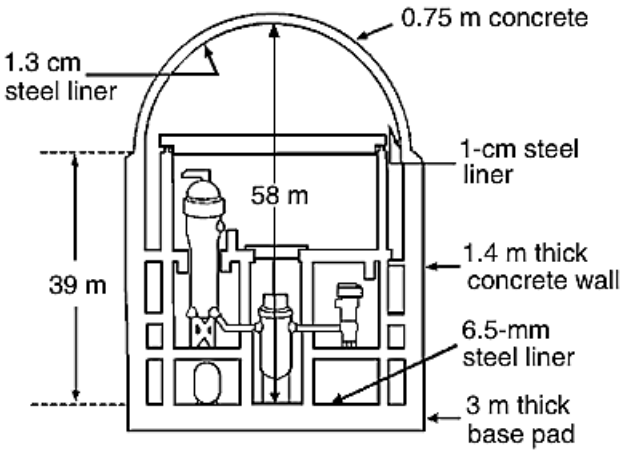


Figure 2-3: Containment Building [4]

Summing up the key components [4] [7]:

- **Reactor Core:** Contains fuel assemblies made of uranium dioxide pellets.
- **Reactor Pressure Vessel (RPV):** It contains reactor core and coolant.
- **Primary Coolant Loop (PCL):** It pumps water of high pressure that will move the heat out of the core to steam producer.
- **Steam Generator:** This passes on heat stored in PCL to secondary loop in which water is now being heated on purpose so that it can turn into steam.
- **Turbine and Generator:** Steam turns the turbines, and thus urges generator connected to it to produce electricity.
- **Condenser:** It sort of cools and condenses steam back into liquid state.
- **Pressurizer:** Maintains the pressure in the PCL to prevent water from boiling.

- **Control Rods:** These consist of mainly boron or cadmium to regulate fission process.
- **Coolant Pumps:** Ensures the continuous circulation of coolant in PCL.
- **Containment Structure:** A shell that prevents radiations form leaking.

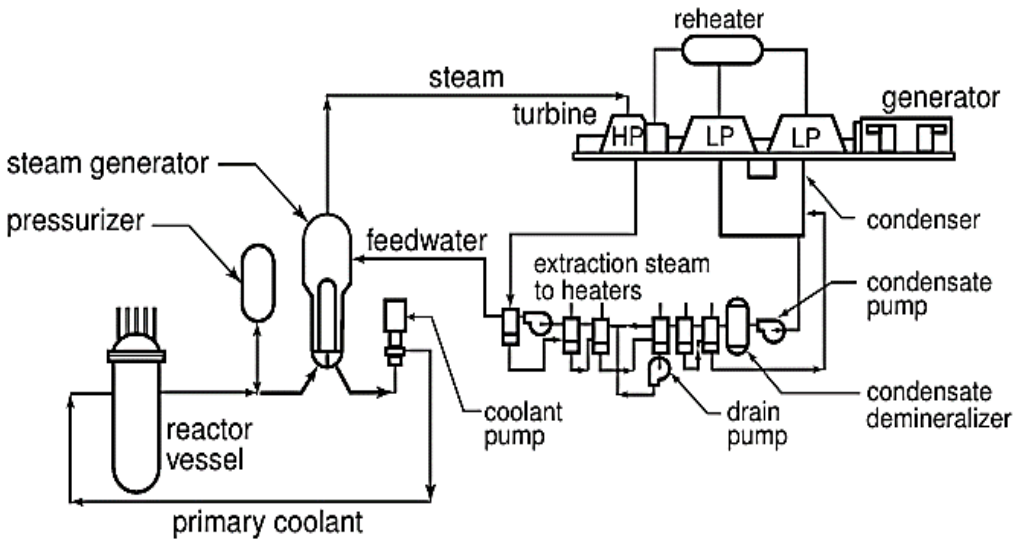


Figure 2-4: Structure of PWR [4]

2.2 PWR Accidents

Examples of accidents covered in Nuclear Power Plant Accident Data (NPPAD) [8] (An Open Time-Series Dataset covering various accidents for NPPs) are: LOCA (Loss of Coolant Accident), SGTR (Steam Generator Tube Rupture), ATWS (Anticipated Transient Without Scram), Rod Insertion and several others. These accident types represent critical fault cases that nuclear power plants are designed to withstand, and studying them is essential for enhancing safety systems and response scenarios. Nuclear accidents are rare but potentially catastrophic events involving the release of radioactive materials with radiations too due to reactor core damage, loss of coolant accident, or system failures. Major nuclear accidents like Chernobyl, Fukushima, and Three Mile Island hints the importance of robust safety protocols and advanced anomaly detection systems.

The NPPAD dataset provides a comprehensive collection of time-series data simulating various accident scenarios in pressurized water reactors. With simulated high-frequency measurements of reactor parameters across multiple accident types, the dataset serves as a valuable resource. It can benefit in generating and testing machine learning models to detect early faults, types of accidents and diagnoses of the system during the operation of the nuclear plant [9]. Accidents discussed in NPPAD are [8] [9] [10]:

- **NORM:** Normal Operating (Not an anomaly but data is generated to keep records from normal operations).
- **LOCA:** Loss-of-Coolant Accident, a critical failure where a reactor's primary coolant is lost due to a pipe break or other rupture. This type of event can lead to the overheating of the nuclear core and potentially a meltdown, as famously occurred at Three Mile Island.
- **LOCAC:** Loss of Coolant Accident - Cold Leg, a severe accident where a large rupture in the cold leg of the reactor coolant system causes rapid coolant loss, risking core overheating.
- **SLBIC:** Steam Line Break Inside Containment, a steam line break within the containment will cause rapid depressurization with difficulties in cooling the core.
- **SLBOC:** Steam Line Break Outside Containment, breakage of a steam line outside the containment poses threat of radioactive steam escape and shock of the system by heat.
- **SGATR:** Steam Generator A Tube Rupture, a tube rupture in Steam Generator A will lead to the leakage of primary coolant into secondary loop.
- **SGBTR:** Steam Generator B Tube Rupture, like SGATR, but happens on the steam Generator B, which jeopardizes exposing the secondary side to radioactivity.
- **RW:** Rod Withdrawal, unintended withdrawal of control rods increases reactivity, possibly causing a power surge.
- **RI:** Rod Insertion, sudden insertion of control rods reduces reactor power rapidly, potentially triggering a transient.
- **FLB:** Feedwater Line Break, a rupture in the feedwater line disrupts cooling, risking steam generator dry out.
- **MD:** Moderator Dilution, the accidental dilution of the neutron moderator reduces reactivity control, increasing power risk.

- **LR:** Load Rejection, a sudden drop in electrical load causes turbine and pressure transients in the reactor system.
- **LLB:** Letdown Line Break in Auxiliary Buildings, the coolant loss and imbalance in the hydrostatic pressure can happen when the sloppy containment of the letdown line is damaged.
- **SP:** Spark Presence for Hydrogen Burn, local combustion hazard, ignition risks are possible due to detection of spark in regions rich in hydrogen.
- **LACP:** Loss of AC Power, alternating current power loss deprives the important safety systems and they have to activated on backup.
- **LOF:** Loss of Flow – Locked Rotor, coolant circulation stops due to a locked rotor in one of the primary coolant pumps, which causes a threat of core overheating.
- **ATWS:** Anticipated Transient Without Scram, a dangerous condition where the reactor fails to shut down automatically during a transient.

Chapter 3: Varied Neural Networks

3. Model Architectures

Various neural network architectures are used for the classification of nuclear power plant (NPP) accidents, and a transformer model is employed for sequential forecasting analysis. The choice of models was due to their ability to cover the time-series characteristics of the data. The work, structure, and the functionality of every architecture used has been addressed.

3.1 Transformers

The transformer, Figure 3-1, is a deep learning AI architecture that deals explicitly with data arising as a sequence (natural language processing, time series processing, and computer vision). An Important aspect of the transformer, which differentiates it from the other frameworks, is its attention mechanism called self-attention. This mechanism enables weighing of relative importance of various components of the parts of input dynamically and parallelly [2] [11].

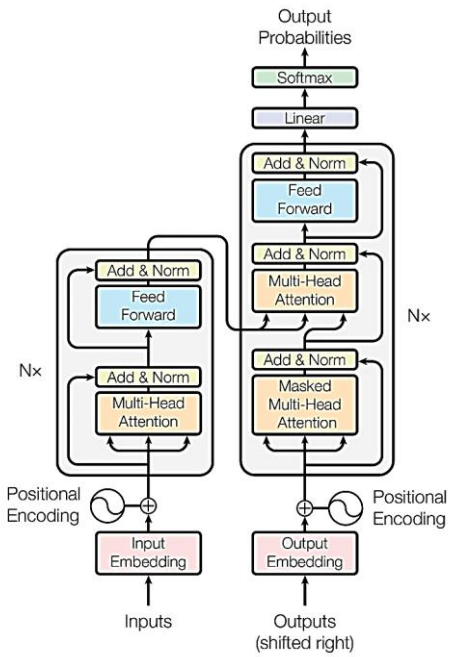


Figure 3-1: Transformer Architecture [11]

3.1.1 Input Embeddings

Input embeddings convert various input tokens such as words, time-series data and pixels into a fixed length vector representation. It can be of any length depending on the desired dimensionality, but it is commonly around 250 dimensions. The primary purpose of input embeddings is to enable the attention mechanism in subsequent steps to capture semantic relationships, average meanings, or calculate static solutions.

Static_Embedding_E (Any word) → Contextual_Embedding_Y (Any word)

Applications of input embeddings include text classification, machine translation, and question answering.

3.1.2 Self Attention

Self-Attention calculates the relationship between every input token, thus capturing the dependencies of one token over the others. The distance between the tokens does not affect this process. Let say we have three words: “Humans love smartphones”. What it will do is take each word and create its input embedding of certain length vector, say E. It then passes this to the attention mechanism block and calculates it contextual embedding, say Y. Any word is linear combination of each word in the sentence.

$$E_{new_{word}} = \sum \alpha_{ij} E_{word} \quad \text{; where } \alpha \text{ represents the attention weights} \qquad (3\text{-}1)$$

First, it separates the Query (Q), Key (K), and Value (V) vectors from the static embeddings. It then calculates the dot products and assigns weights to obtain the contextual embeddings.

$$Attention\ (Q, K, V) = softmax\ \left(\frac{QK^T}{\sqrt{d}}\right) V \qquad (3\text{-}2)$$

Right after applying the softmax we calculate the product (dot) of the W vectors with the remaining value ones, and finally, contextual embedding are generated. This process is parallelized and performed using matrix operations. Once the Q, K, and V vectors are generated from the static

embeddings, alongside a new matrix of same dimensions is also generated accordingly. For each matrix these are the weights that get updated while back-propagation.

3.1.3 Multi-Head Attention

It is where more than one self-attention heads are used parallelly and are combined together to form what is known as multi-head attention. It increases the number of learnable weight matrices to allow multiple pairs of Q, K, and V vectors for each word. Let us take the words money and bank as an example, multi head attention does calculations as:

Money → Self Attention 1 and 2 → Y (1,2)

Bank → Self_Attention 1 and 2 → Y (2,1)

$$Multihead\ (Q, K, V) = Concat(head_1, head_2, \dots\dots\ head_n)W^0$$
$$where\ head_i = Attention\ (QW_i^Q, KW_i^K, VW_i^V) \quad (3\text{-}3)$$

3.1.4 Positional Encodings

When self-attention is doing calculations in parallel, the orders of words is lost. We have to keep track of the order and arrangement of given words or embeddings. This means we need to assign positions to each of the embeddings. As mentioned in the original paper on transformers, we require bounded, continuous periodic functions. Sine and cosine come to mind when we talk about the said properties.

$$Positional\ Encoding\ (pos, 2i) = \sin \frac{pos}{10{,}000^{\frac{2i}{d_{model}}}} \quad (3\text{-}4)$$

$$Postional\ Encoding\ (pos, 2_{i+1} = \cos \frac{pos}{10{,}000^{\frac{2i}{d_{model}}}} \quad (3\text{-}5)$$

Where i: dimension and pos: position

Our model does not involve any recurrence or convolution. So as to have the model utilize the order of sequence there is certain information that we have to inject at the relative or absolute position of the tokens in the sequence and accordingly to the situation in Figure 3-2.

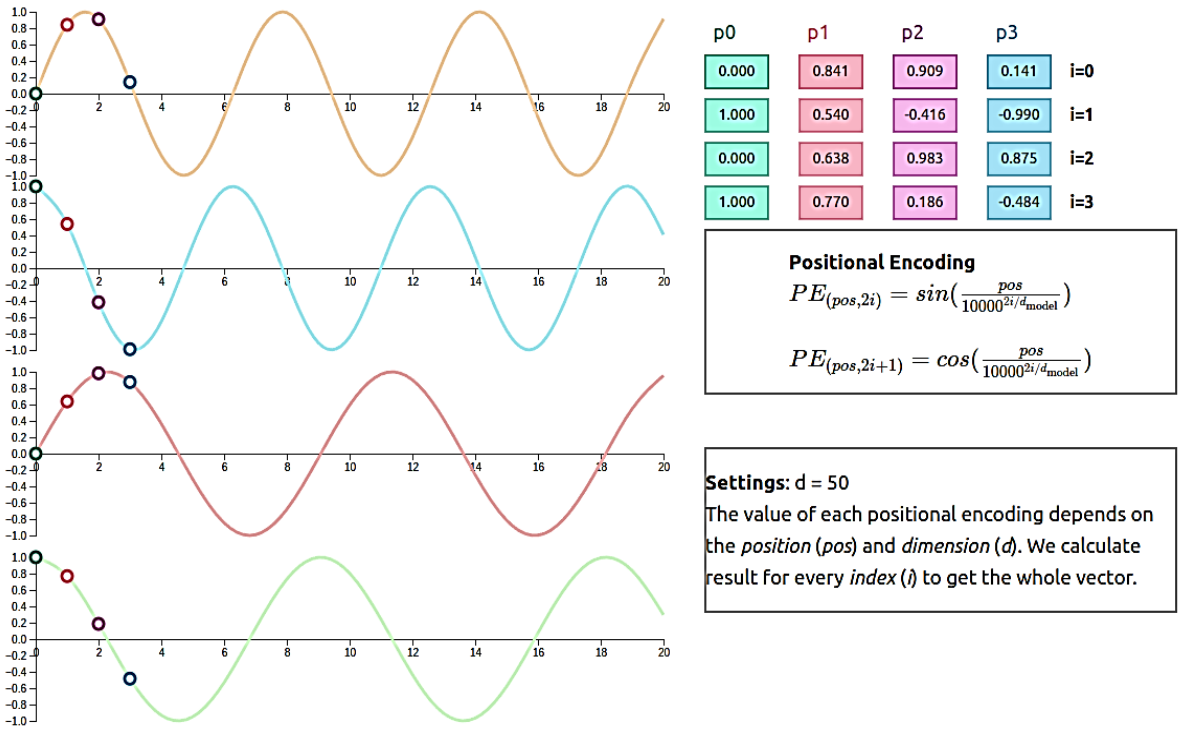


Figure 3-2: Positional Encodings [31]

- The problem with the Sine problem here is that it can give same value for different positions, so we use cosine accordingly to create vectors with two values for the same position.
- Even if it repeats itself, we just in increase the number of pairs of sine and cosine.
- Dim (positional vector) = dim (embedding).
- Add the embedding and PE vector.
- PE are generated 2 by 2 from graph's pairs according as follows:

$$i = \frac{d_{model}}{2} - 1 \tag{3-6}$$

3.1.5 Masked and Cross Self Attention

Ensuring the model generates predictions in an auto-regressive manner i.e., one token by one token predictions, Figure 3-3, while ensuring it cannot see future tokens during generation.

$$Mask\ (i,j) = 0\ if\ i \geq j\ (access)\ and \infty\ if\ i < j\ (not\ allowed)$$

Concurrently, the cross attention facilitates the attendant process of this model to note differing schemes of the input to yield the output scheme.

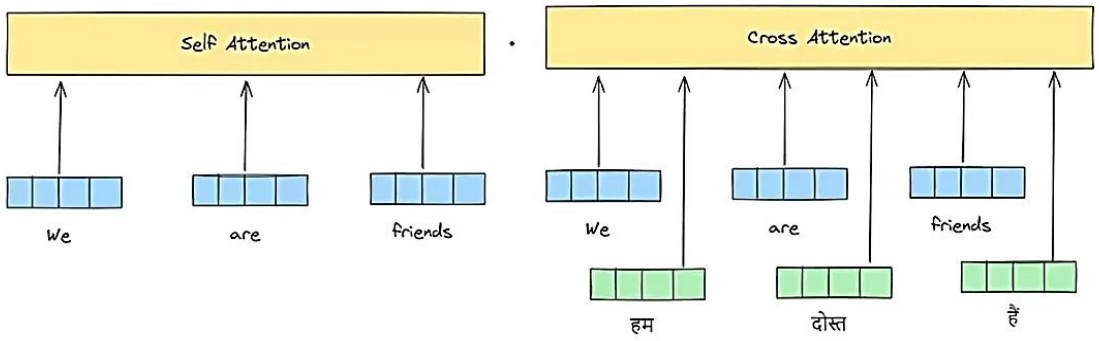


Figure 3-3: Cross Attention [32]

3.1.6 Layer Normalizations

Layer normalization is performed to alleviate training instability, allow faster convergence, reduce internal covariate shift, and regularization. It is a valuable method in transformer models used to both stabilize and fasten convergence since it normalizes the value of every inputs of every level. It makes sure that the model processes all the information uniformly with distributions of the input. Consider that in the process of training a neural network because you are changing the weights at some point you might end up having large values of the weights. As soon as this occurs, the activations also get large which thereby makes it difficult to train and learn efficiently. This brings about training issues. This can be remedied by normalizing its activations so that the activations are in a fixed range. This is not only less unstable during the training process, but also faster, and accordingly the model can learn more rapidly. Normalization like batch normalization helps to regularize the model as well thus modeling in new data can be used better [12]. The system of batch normalization functions in the following way:

$$Normalized\ Value = \frac{orginal\ value - \mu}{\sigma} \tag{3-7}$$

$$Final\ Value = (Normalized\ Value * \gamma) + \beta \tag{3-8}$$

Here, γ and β are initially set to 1 and 0, respectively, but are adjusted during training.

3.1.7 Encoder

Transformer encoders are a significant component of encoder-decoder architecture. The important role of encoder is to process and encode input sequence in the format which can be understood by the model. Self-attention is the main part of the encoder block because this is where the contextual embeddings are generated and are normalized using batch normalization, along with the help of residual connections. Residual connections directly pass the original embeddings forward to interact again with newly generated embeddings to calculate relationships. The encoder is made up of successive layers, each of which could have an attention block and a feed forward or simple neural network, Figure 3-4, and data is passed through neurons and their weights updated on back propagation. This feed forward assists the model to learn on high features of the input. ReLU activation function is used to prevent negative values to be applied and convert them to zero without altering positive values of the network. [13].

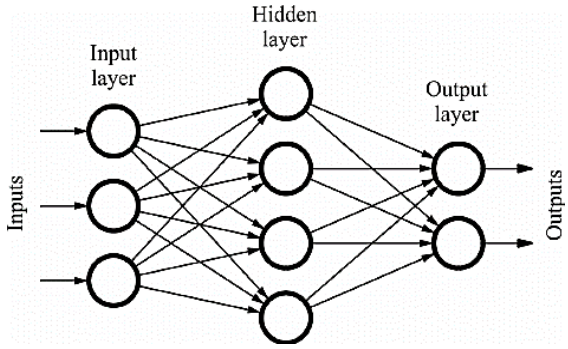


Figure 3-4: Feed Forward Neural Network [33]

3.1.8 Decoder

Right after the encoder, the transformer architecture also uses decoder which is another important component that is used in the generation of necessary output sequences. After the encoder has processed the input sequence and generating hidden contextual information environment which is not suitable to generate final output ─ the decoder takes over. The decoder uses these hidden states as its starting point and generate the final sequence one token at a time. It also considers the previously generated tokens. The information in decoder part is processed as follows [13] [14].

- The outputs are first produced into the decoder block and are passed through the masked multi-head attention in attempt to prepare the embeddings that awaiting encoder output. Residual connection is a part of this step as well.
- Once the output of encoder arrives, it relates to the results already obtained by the decoder and tries to learn the patterns.
- Linear Layer: To apply a linear transformation to the output of encoder, it is passed through a linear layer. This linear layer is the feed forward neural network which contains weight matrix and bias vector.
- Softmax function: After passing through the linear layer, the results are sent to the softmax activation function to produce output values that sum to 1. Each of these values represents the probability of a particular class.
- Prediction and training: Prediction of the next token is done using output probabilities. The group which is most likely is chosen and the new token is put in the sequence. It is through this that the transformer has learnt patterns and how it works in inference. We are able to adjust the wanted hyperparameters to make the model amalgamate with the data and then implement it to the activities like machine translation.

3.2 Multi-Layer Perceptrons

Neural networks (NN) refers to the machine learning models which are based on the structure of a human brain and the functioning that of a human brain, Figure 3-5. These are constituted by layered arrangements of interconnected nodes (neurons) that process and attempt to map the input information towards learning patterns of complex information and they are applicable in classification, regression and forecasting of time series. They are able to work even with large pieces of data, manipulate them and give out results that are desired. The principal notion of neural networks is that there is a chance to learn an approximation of any entity f so that [15]:

$$y = f(x) \tag{3-9}$$

Neural Networks are very good with recapturing the meaning of very complex or imprecise information and can be trained to find out patterns or notice trends that are too complicated to be observed by a human. Neural networks have two major advantages:

- **Adaptive learning:** A learning approach where the learning rate or model parameters are adjusted automatically during training based on the performance or behavior of the model.
- **Self-Organization:** NN is able to come up with their own internal arrangement or a form of the information passed on to them during training.

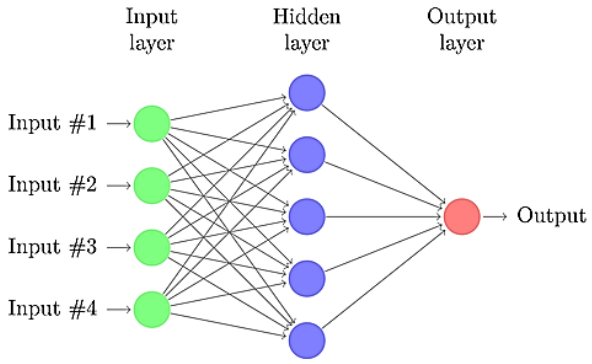


Figure 3-5: Neural Network [17]

3.2.1 Single Neuron

Basic unit of a neural network is neuron although it may also be known as node or unit. It receives input of the form of other units or directly as received by an external source and offers an output. Each of the inputs has a weight (w) which denotes its significance among the rest. These inputs are then multiplied weighted according to weights and the final output calculated by utilizing an activation function (f), as the output axon in Figure 3-6 shows.

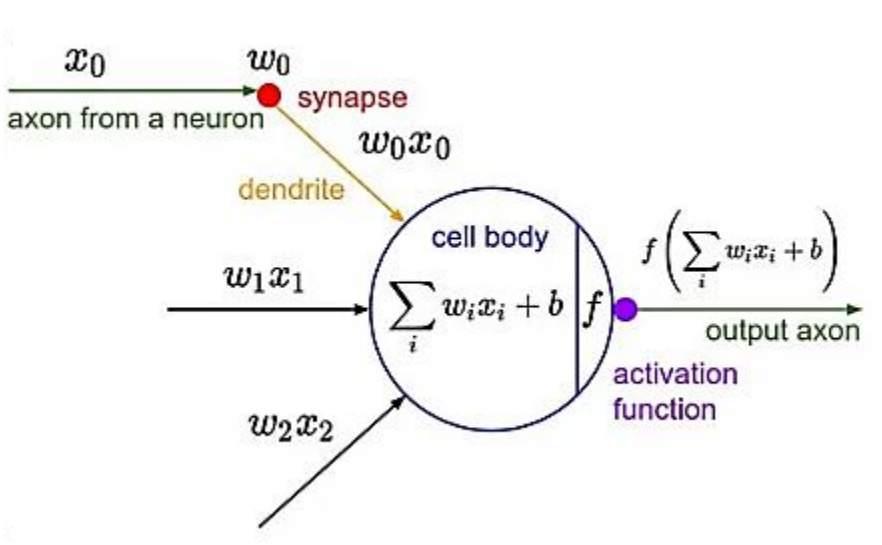


Figure 3-6: Neurons and weights [34]

The network accepts two numerical inputs, x1 and x2, and is characterized by so-called weights, i.e., w1 and w2, which are associated with input.

3.2.2 Activation Functions

The Function f is a non-linear one and is referred to Activation Function. Activation function is used in introducing non-linearity to the output of a neuron. It is significant since majority of the information that we get out there in the world is nonlinear and we would want neurons to learn nonlinear representation.

Each activation function (or non-linearity) operates on one number, and discards or updates all the other numbers by dropping or updating them based on a previously quantified mathematical operation. Various activation functions There exist a number of activation functions [16]:

- **Sigmoid:** It has real-valued as its input and transforms it to a range between 0 and 1. It is predominantly applied in the determination of binary classifications.

$$\sigma = \frac{1}{1+\exp -x} \tag{3-10}$$

- **Softmax:** When the task is multi-class classification, then in the output layer of multilayer perceptron, softmax is employed. That is most preferably served by a non-linear activation.

Softmax takes an arbitrary vector with any arbitrary values and compress them in the range between 0 and 1, in such a way that the sum is always 1 or 100 percent.

$$\sigma(z)_i = \frac{e_i^z}{\sum_{j=1}^{k} e_j^z} \tag{3-11}$$

- **Tanh:** It also just takes a real-valued input and it is scaled somewhere between -1 to 1.

$$\tanh(x) = \frac{2}{1+\exp(-2x)} - 1 \tag{3-12}$$

- **ReLU:** Rectified Linear Unit, is a real-values input function which is thresholded at zero (substitutes negative values with zeros)

$$f(x) = \max(0, x) \tag{3-13}$$

3.2.2.1 Bias

A neural network bias is one of the parameters which actually can be trained and is associated with the weighted combination of the inputs before going to the activation function. This enables the activation function to be left shifted or right shifted aiding the model to fit the data better because it can now be used to model patterns that are not going through the origin.

3.2.3 Propagations

Propagation in neural networks refers to the process of passing data through the network.

3.2.3.1 Forward Propagation

Feeding the data into the network is done layer by layer and weighted sum along with bias, and activation functions are computed by each neuron in the network and the result transfers to next layer. Let say we have two inputs numbers: 35 and 67 in Figure 3-7.

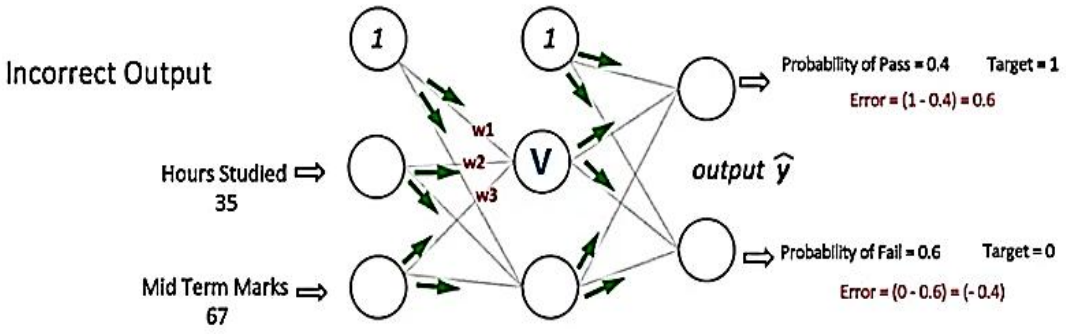


Figure 3-7: Forward Propagation [17]

3.2.3.2 Backward Propagation

After error has been computed at the output layer of the network, it is then fed back through the network to correct the weights and biases in the network, Figure 3-8, by means of optimization techniques such as gradient descent. That is, the back propagation is implemented by using the total error at the output nodes to derive the gradients. The weights in the network are fine-tuned with the final objective of ensuring that the discrepancy between the predicted value, and the actual value, is decreased.

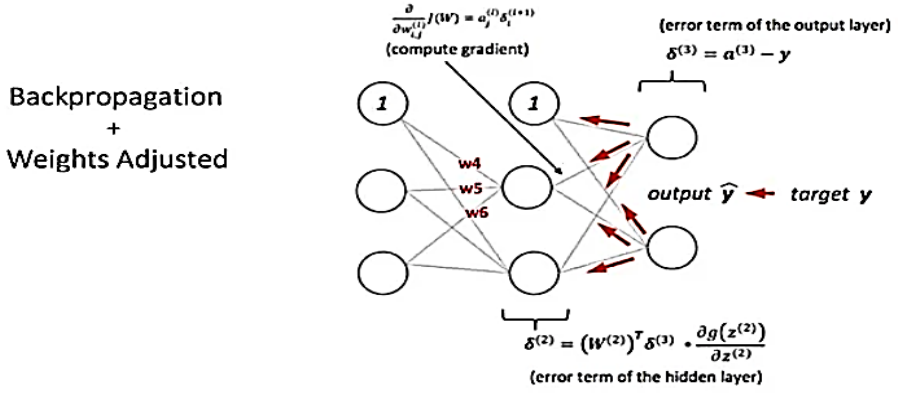


Figure 3-8: Back Propagation [17]

Neural networks work by processing data through a layered structure of interconnected nodes. It begins from the input layer that takes in unaltered information to the network. Neurons in further hidden layers take inputs and apply them to weights corresponding to them and a bias is added to them before they are passed through an activation function that introduces non-linearity. This forward pass is carried out in a layer by layer until data reaches the output layer that makes the final prediction or classification. Once the output of the model is compared with the actual value, the error is computed and back to the network through the backward propagation. During this step, the network updates its weights and biases to minimize the error using optimization algorithms like gradient descent. This cycle repeats across multiple iterations, called epochs, until the network learns to make accurate predictions with a significantly reduced error.

3.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a form of neural network that use a grid like symmetry of the data to do the processing, like an image. CNNs do so by automatically learning and extracting spatial features (edges, textures, shapes etc.) inside convolutional layers using filter (kernels) that slide across the input [18].

A typical CNN consists of several key components:

- Convolutional layers, that feature extraction.
- Non-linearity through the use of activation functions (such as ReLU).
- Spatial dimension reduction through pooling layers.
- Fully connected layers to make final predictions.

CNNs are widely used in solving image classification problems, as well as object detection and other pattern recognition issues. CNNs consume images and process them as tensors because they have the capacity to capture characteristics of hierarchies and spatial dependencies. Tensors are numeric matrices that have extra dimensions. Figure 3-9 shows feature learning, edges, curves, lines, and shades are all learned using convolutional layers, ReLU activation, and pooling layers.

- Classification: Fully connected layer (FC) assigns a probability to the object in the image, indicating how likely it is to be what the algorithm predicts.

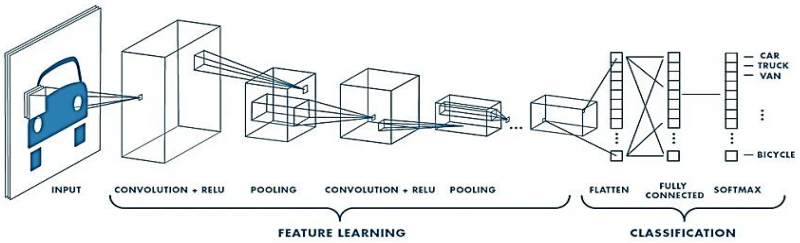


Figure 3-9: CNN Working Flow [19]

3.3.1 Convolutional Layer

Any picture may be expressed as the set of the values of pixels as the Figure 3-10. As an illustrating example, a 5 x 5 grayscale image can be represented as a matrix, the elements of which are pixel values which are usually between 0 and 1 (black being 0, and white being 1).

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Figure 3-10: Input Image [21]

Filter: This image is then multiplied by a filter in order to have the desired convolutional layer. These filters, Figure 3-11, are also matrices and may differ in shape with values to extract different features such as edges, curves, lines. Kernel is another name for a filter or feature detector.

1	0	1
0	1	0
1	0	1

Figure 3-11: Kernel [21]

All the values (in each pixel) of the image in input gets multiplied by the corresponding value in the filter to form the convolution layer. Conv layer is sometimes referred to as convolutional feature, feature map or filter map. The result of the convolved feature, Figure 3-12 (the output after applying a filter to an image) depends on three important settings that are chosen before starting [19]:

- **Depth:** In our example, we used a simple image with depth = 1 (like a black and white image). But most real images have depth = 3, because they have three color channels: Red, Green, and Blue.
- **Stride:** Stride is the count of movements that the filter makes one time. When stride = 1 the filter will be stepped one pixel at a time. When its stride is 2, then it traverses two pixels per step. The bigger the stride, the smaller the output image (feature map).
- **Padding:** Padding refers to the addition of pixels (normally zeros) on the image sides. This is in order to avoid the size of output being so small. In the majority of the cases, 1 pixel is inserted everywhere.

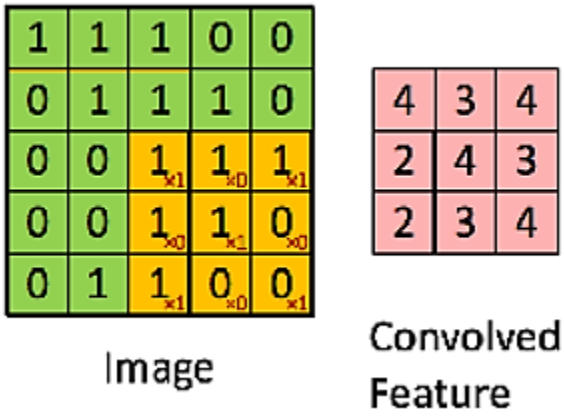


Figure 3-12: Striding Kernel [21]

3.3.2 Pooling Layer

Here the feature map is downsized with the significant information being retained. This is termed as pooling and it is also referred to as down sampling or subsampling. Pooling can be of a variety; there is max pooling and average pooling and sum pooling. However, in most cases, max pooling is used, Figure 3-13, which picks the highest value from a group of nearby pixels [20].

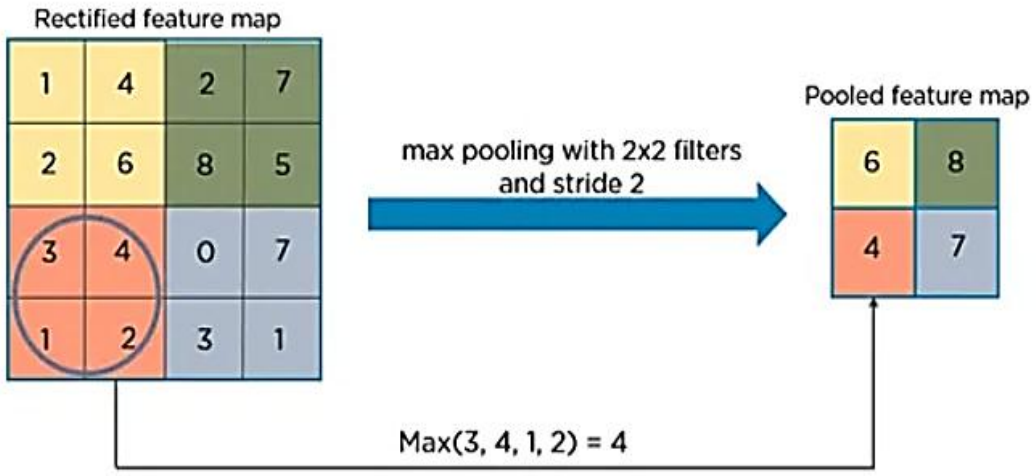


Figure 3-13: Max Pooling on ReLU Feature Map [21]

3.3.3 Fully Connected Layer

Fully Connected layer is like a typical neural network where each neuron in one layer is connected to each neuron in the other layer. It also applies a special weighted operation at the end of it known as Softmax to make the final decision. The previous layers (convolution and pooling) assist in discovering significant items in the picture. These features are then used in the Fully Connected layer to come up with what the image is representing, through what it has come to know through training. The output of this layer shows probabilities for each class (like "cat", "dog", etc.), and all the values add up to 1. This happens because the Softmax function converts the scores into values between 0 and 1, making it easy to identify which class the model considers most likely [15].

3.4 Recurrent Neural Networks

RNN stands for Recurrent Neural Network. It is one of the main types of a neural network that is capable of remembering information of past steps. This is why it is suitable to sequential data like text, speech or time-series. Real-life applications include Siri on Apple products, Voice Search by

Google, all of which are RNNs. Recurrent Neural Network (RNN) resembles to feed-forward neural network that has been improved with memory. It is called recurrent because it applies the same process to all the inputs in a series, and the output is determined by the present input coupled with what it has learned on previous inputs. After generating an output, it sends that output back into the network to assist with the next step. Unlike regular neural networks, where each input is treated separately, RNNs remember past information, making them ideal when dealing with something such as speech recognition or handwriting recognition where data order is important [22].

RNNs possess a sort of memory which helps them recall previous inputs. When making decisions an RNN does not only take into consideration the incoming input but what it has learnt about previous inputs. The outcome of the final step serves as an input in the next step and this forms a feedback loop. This means that RNN uses current state and the new input to update the current state using the next state. On this basis, the information keeps circulating the loop. Concisely, RNN has two inputs, which are present and the immediate past. This is significant since the sequence in which the data is presented is relevant, and RNNs are able to make use of the same in predicting what is likely to occur next, which is a capability of many other models.

3.4.1 RNN Types

RNN is a kind of neural network that can memorize the previous outcomes and apply to the subsequent ones. They are mostly used in language tasks and speech recognition because they handle sequences well [23]. They have their types too as the arrangement in Figure 3-14 shows.

- **One to One RNN:** One input and one output. Used in simple tasks like image or number classification.
- **One to Many RNN:** One input gives multiple outputs over time. For example, generating a sentence from a single image.
- **Many to One RNN:** Many inputs give one output. Used for tasks like determining the sentiment of an entire sentence (sentiment analysis).
- **Many to Many RNN:** Multiple inputs give multiple outputs. Useful for tasks like translating one sentence into another language.

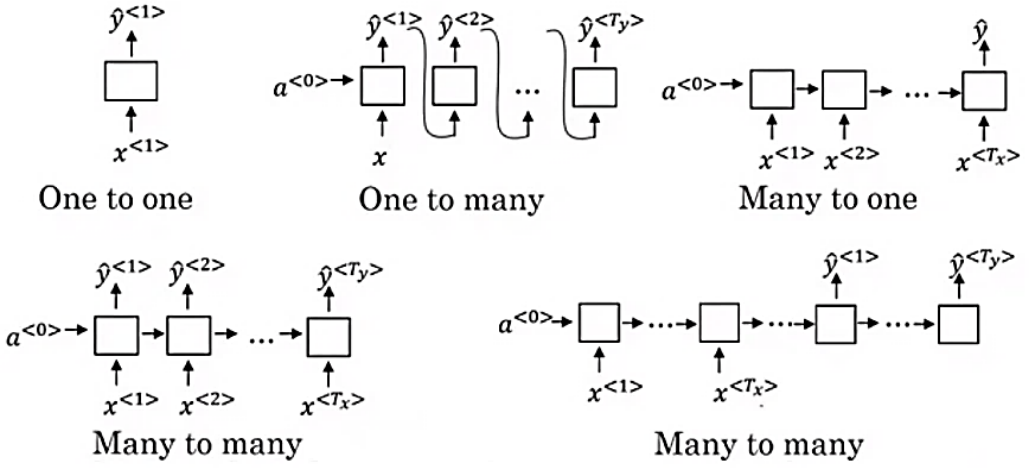


Figure 3-14: RNN Types [35]

3.4.2 Gradient Problems in RNNs

When training Recurrent Neural Networks (RNNs), the gradient can sometimes become either too large or too small. This makes learning difficult and causes problems such as:

- Poor performance
- Low accuracy
- Very slow training

Exploding Gradients: This occurs when gradients become excessively large, causing values to grow out of control and making the model unstable. Solutions include identity initialization, truncated backpropagation, or gradient clipping to limit large values.

Vanishing Gradients: This happens when gradients are extremely small whereby the model will not learn at all or learn at a very slow rate. Remedies consist in the initialization of the weight proper or the utilization of suitable activation functions or employing a network LSTM (Long Short-Term Memory) developed to counter this issue.

3.4.3 Long Short-Term Memory Model

RNNs can't remember information for long periods and tend to forget what they learned earlier. LSTM networks are applied to remedy this issue of vanishing and exploding gradients problem. They help with learning short-term dependencies more effectively. Additionally, when new information is introduced, RNNs completely change the old information and struggle to distinguish what is important. In contrast, LSTMs modify only a small part of the old information. It has special gates as the Figure 3-15 shows that control how information flows and what to keep or forget.

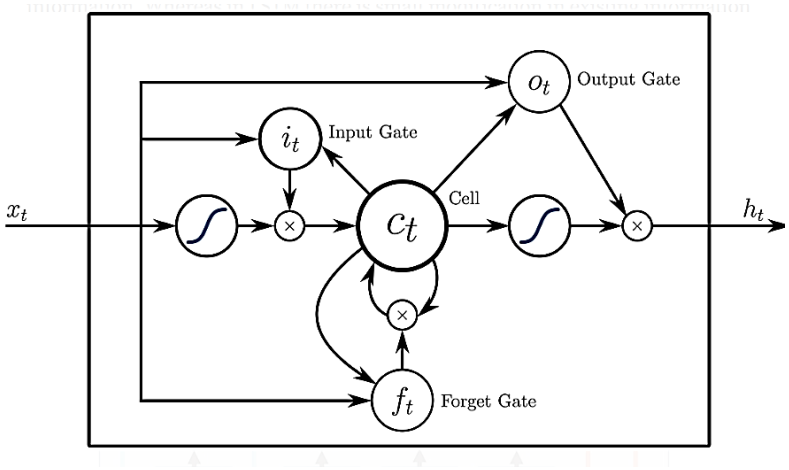


Figure 3-15: LSTM [35]

The gates dictate on what information is to be cherished and stored in future and which information should be destroyed. It consists of three main gates such as input gate, output gate and forget gate.

- **Forget Gate:** It is the gate that determines the kind of information to remember and the kind to forget. It removes unnecessary information from the neuron's memory to help the model work effectively. The gate gets two inputs — one from the previous cell's output and one from the current input. These inputs are combined with weights and bias, Then the results sent to a sigmoid function. Sigmoid generates a value between 0 to 1. If the value

is 0, said gate ignores the information. If it's 1, then it needs to be retained because it is relevant.

- **Input Gate:** This gate determines what new information is to be incorporated in the memory of the neuron. It selects values to be added using the sigmoid function. It next compiles a list of new data with the tanh function that returns -1 to 1. Sigmoid is a filter that trims the information that is to be taken into the memory cell.
- **Output Gate:** The significant information in the current memory cell is passed through and that makes the particular gate. It scales the values between -1 and 1 using the tanh function. It also uses previous output and present input and based on the sigmoid function, determines values to be output and these values are thus outputted as the final output.

A Recurrent Neural Network (RNN) processes data one step at a time, using a special memory to keep important information from earlier steps. At each step, it combines the current input with what it remembers from previous steps to determine the output. The output from one step is also fed back into the network to help with the next step. This allows RNNs can understand sequences such as sentences or time-based data, where the order and context are important. However, RNNs can sometimes forget important details after many steps. To address this, more advanced models like LSTM are used, which are better at retaining and managing information over longer sequences [22] [23].

Chapter 4: Setup, Results, and Analysis

4. Methodology and Results

4.1 Dataset Preparation

The NPPAD dataset was downloaded from the GitHub repository thu-inet/NuclearPowerPlantAccidentData/tree/main. This repository contains two main folders: dose_csv_data and operational_csv_data, along with several Python scripts for data preprocessing and installing dependencies. However, for this project only the operational data was used. The downloaded data is organized in a parent directory that contains 18 subdirectories, each named after a specific type of nuclear power plant accident. Inside each of subdirectory, there are 100 CSV files containing reactor readings related to that particular accident. These readings are useful for identifying and classifying the accident type [24]. However, a few accidents like NORM, FLB, and TT have only one CSV file each. Therefore, they were excluded from the classification task but still considered for the sequential analysis part of the project.

parent_directory/accident_type_directory(x18)/1.csv, 2.csv, 3.csv, …, 100.csv

4.1.1 Operational Data Preprocessing

The data preparation coding pipeline was responsible for loading and preparing sequential data from the operational CSV files of the NPPAD dataset. It iterated through each subdirectory within the main data directory. Each subdirectory corresponds to a specific type of nuclear power plant accident and contains CSV files with sensor readings. Each CSV file includes 96 reactor parameters, capturing measurements such as temperature, pressure, coolant levels, flow rates, control rod positions, and radiation measurements. These features represent the operational state

of a nuclear power plant during different accident scenarios. For each CSV file, the data preparation coding pipeline, Figure 4-1, did the following [25]:

- Imports required libraries: os for directory operations, pandas for data handling, and numpy for numerical operations.
- Defines a function load_sequence_data (root_dir, seq_length=96) to load and process the data.
- Reads the .csv file into a table (Pandas Data Frame).
- Initializes x and y as empty lists to store input sequences and labels.
- Creates a label_map to create custom label indexing.
- Initializes a new_label counter, starting from 0.
- Iterates through each subdirectory in root_dir path, where each subdirectory represents a different accident type.
- Removes the TIME column if it exists and keeps only rows where TIME ≥ 20.
- Verifies that each file has exactly 96 features (columns). If a file has more, it trims the extra ones. If it has fewer, the file is skipped.
- Extracts the first 96 rows of the data (a sequence of readings) and stores it if the shape is exactly (96 rows x 96 columns).

The pipeline assigned a label to each sequence based on the accident type. Some accident types that had only one CSV file were skipped to avoid issues during classification. After execution, we got:

- 1193 valid sequences, each of shape (96, 96) — representing 96-time steps with 96 features at each step.
- A total of 1193 labels, one for each sequence.
- 12 unique labels representing 12 distinct accident types after removing the ones with only 1 csv file.

```
[ ]: #-----------------------------------------------------
     # IMORTING CSV FILES AND PERFORMING REQUIRED OPERTIONS
     #-----------------------------------------------------

     import os
     import pandas as pd
     import numpy as np
     def load_sequence_data(root_dir, seq_length=96):
         X, y = [], []
         label_map = {}
         new_label = 0
         for label, folder in enumerate(sorted(os.listdir(root_dir))):
             folder_path = os.path.join(root_dir, folder)
             if os.path.isdir(folder_path):
                 csv_files = [f for f in sorted(os.listdir(folder_path)) if f.endswith(".csv")]
                 if len(csv_files) <= 1:
                     print(f"Skipping folder {folder}: Only {len(csv_files)} CSV file(s) found.")
                     continue
                 print(f"Processing folder: {folder}")
                 label_map[label] = new_label
                 new_label += 1
                 for csv_file in csv_files:
                     csv_path = os.path.join(folder_path, csv_file)
                     try:
                         df = pd.read_csv(csv_path)
                         if "TIME" in df.columns:
                             df = df[df["TIME"] >= 20]
                             df.drop(columns=["TIME"], inplace=True)
                         if df.shape[1] > 96:
                             print(f"Fixing {csv_file}: Found {df.shape[1]} features, trimming extra columns...")
                             df = df.iloc[:, :96]
                         elif df.shape[1] < 96:
                             print(f"Skipping {csv_file}: Expected 96 features, found {df.shape[1]}")
                             continue
                         if len(df) >= seq_length:
                             seq = df.iloc[:seq_length].values
                             print(seq.shape)
                             if seq.shape == (seq_length, 96):
                                 X.append(seq)
                                 y.append(label_map[label])
                             else:
                                 print(f"Skipping sequence in {csv_file}: Shape mismatch {seq.shape}")
                     except Exception as e:
                         print(f"Error processing {csv_file}: {e}")
         if X:
             X = np.array(X, dtype=np.float32)
         else:
             print("No valid sequences found!")
             return None, None
         y = np.array(y, dtype=np.int64)
         print(f"Final Data Shape: {X.shape}, Labels Shape: {y.shape}")
         print(f"Unique Labels After Mapping: {np.unique(y)}")
         return X, y
     X, y = load_sequence_data("Operation_csv_data")
     if X is not None:
         print(f"Data shape: {X.shape}, Labels shape: {y.shape}")
     else:
         print("Error: No valid data loaded.")
```

Figure 4-1: Data Preparation Pipeline [25]

4.2 Model Implementations

The data has been divided into three such groups as training, validation, and testing before the models start being trained. Training data was of shape (954, 96, 96) i.e. 954 sequences of 96-time steps and 96 features. Validation set that was used had 119 such sequences and the test set had 120 sequences. All the values were scaled between 0 and 1 after the split using the min-max normalization. This was done by only considering the minimum and the maximum values of the values of the training set, so as to overcome the data leakage. The same scaling was then applied to the validation and test sets. As a result, the training data was scaled from 0.0000 to 1.0000, while the validation and test sets had values ranging from 0.0005 to 1.0000 and 0.0007 to 1.0000, respectively. This normalization helped the models train more efficiently and ensured all features contributed equally [26].

4.2.1 MLPs

Model 01 and Model 03 were trained for 250 epochs because their training was stable and the loss kept improving. In contrast, Model 02 and Model 04 showed unstable training behavior, thereby necessitating early stopping around 80 epochs. To help with stability, various optimization strategies were applied. All models except Model 04 employed the pure Adam optimizer that involved a fixed learning rate of 0.001. However, Model 04 showed better and stable loss curves when trained using a learning rate scheduler based on ExponentialDecay. This approach helped mitigate sudden spikes or fluctuations in the training loss plotted and shown in Figure 4-2 [27].

Model Performance Summary:

- **Model 01: Single Layer MLP**
    - Test Accuracy: *92.50%*
- **Model 02: Two Layer MLP**
    - Test Accuracy: *89.17%*
- **Model 03: Four Layer MLP**
    - Test Accuracy: *89.15%*

- **Model 04: Four Layer MLP with Batch Normalization (BN)**
    - Test Accuracy: *87.50%*

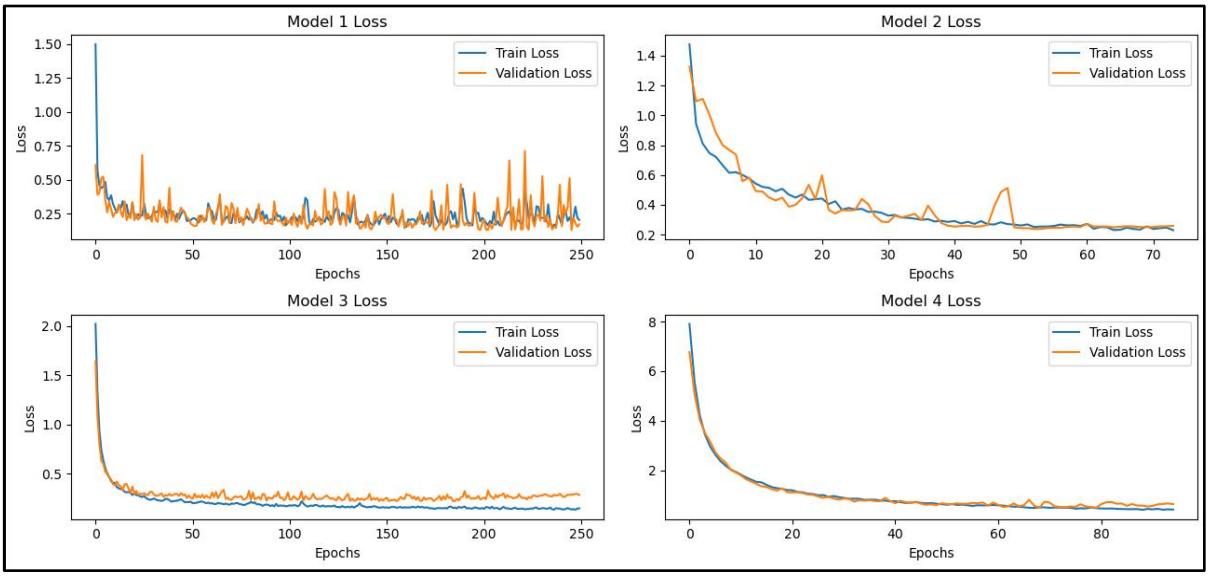


Figure 4-2: MLPs Loss Curves [26]

4.2.2 CNN

Before applying a CNN, the input data was reshaped to include an additional dimension (called the channel). Since the original data was 2D (96×96), it was reshaped to (96, 96, 1) to represent a single-channel input similar to a grayscale image. This reshaping was necessary because CNNs expects input with 3 dimensions: height, width, and channels. The CNN model used for this task consisted of three convolutional layers.

The architecture of the model is as follows:

- **Input Layer:** Accepts input of shape (96, 96, 1)
- **Conv Layers:** 32, 64, and 128 filters three layers each, where Conv layer has a kernel of 3 by 3, and uses ReLU as activation.

- **MaxPooling Layer:** The size of the feature map reduces.
- **Dense Layer:** It is a total connectivity layer that consists of 128 neurons and ReLU activation.
- **Output Layer:** Fully-connected 12-neuron layer (one neuron per each of the classes), where in softmax activation will be used to generate the probabilities of the classes.

The CNN model performed reasonably well. But some fluctuations were noted in the accuracy curves in Figure 4-3, for both training and validation. Despite this instability, the test accuracy reached approximately *75.83%*, showing that the model was still able to generalize fairly well.

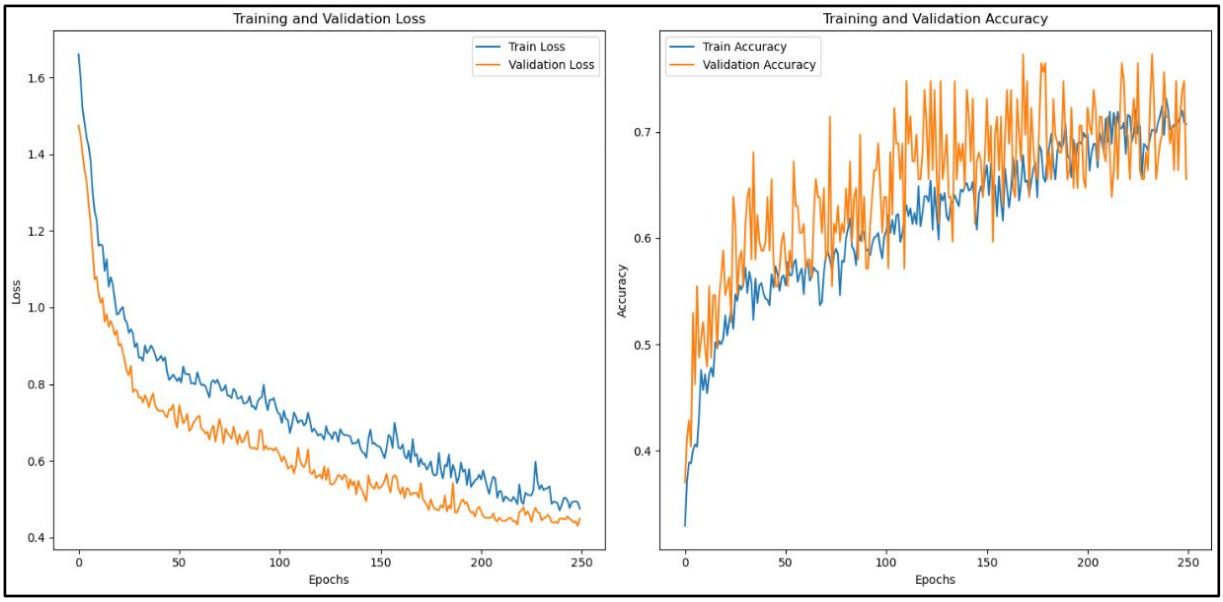


Figure 4-3: CNN Loss and Accuracy Curves [26]

4.2.2.1 Dense Net

DenseNet was designed for classifying data with a shape of 96×96×1. It started with a basic convolution layer to process the input, followed by four dense blocks. Each block contained two convolution layers and connected (concatenates) their outputs with the original input of that block. After every block, max pooling was used to reduce the size of the data, helping the model focus on the most important features. In the end, the model used global average pooling to flatten the

feature maps, then passes the output through a dense layer with dropout to prevent overfitting. And finally, used a softmax layer to output the class probabilities. Test accuracy got stuck around *54–56%*, despite training for 250 epochs as the plot in Figure 4-4 shows. Loss Curves (left plot): Training and validation loss steadily decreased, which is good and showed that model was learning. Accuracy Curves (right plot): Training and validation accuracy both improved over time but plateaued around *~54–60%*, and there was some noticeable fluctuation, especially in validation accuracy.

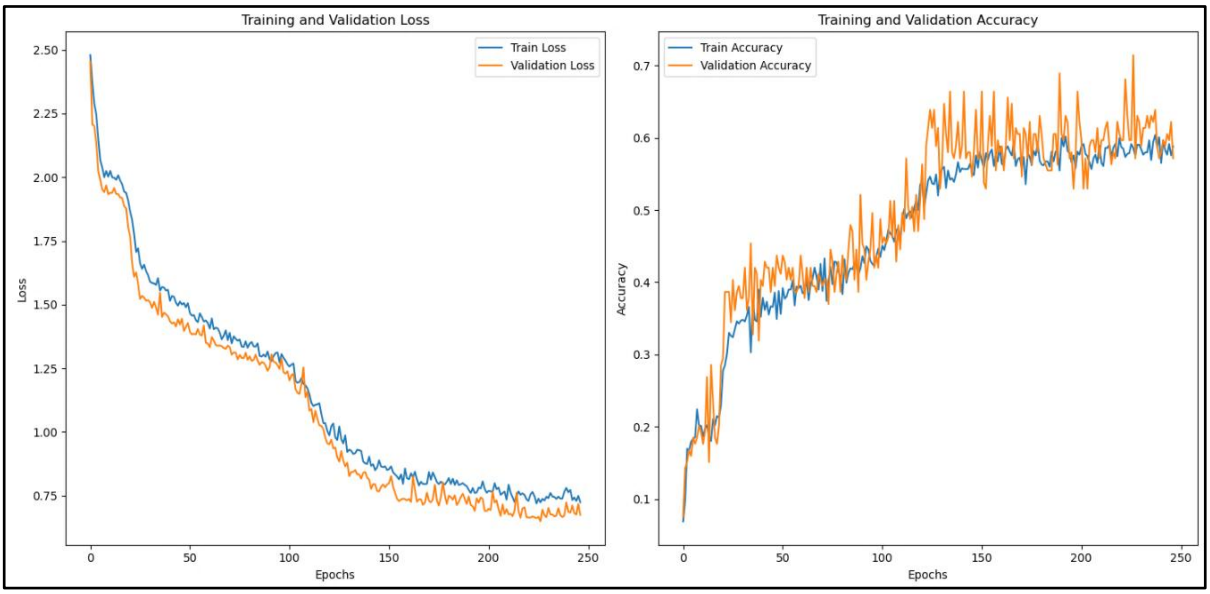


Figure 4-4: DenseNet Loss and Accuracy Curves [26]

Possible reasons for limited accuracy (*~ 54%):*

- **Data Complexity:** Some features might be irrelevant, noisy, or redundant, affecting learning quality.
- **Chunked Sequences:** Each class corresponds to a 500-row chunk from individual CSVs. While this provides structured input, some sequences may not contain enough class-distinctive features, or there may be overlap across classes.

- **Lack of Temporal Awareness:** CNNs are not well-suited for capturing time-based patterns.
- **Model Capacity:** With 04 blocks, the model might either:
    - Not be deep or expressive enough to learn more complex features.
    - Or, if too deep, it may overfit.
- **Batch Size / Learning Rate / Regularization** experimentation**.**

4.2.3 RNNs

- **Model 01: GRU-Based Model with Two GRU Cells**
    - The first GRU layer processes the input and passes the sequence forward.
    - The second GRU layer further processes the output.
    - Dropout is added, followed by a dense layer with softmax to make predictions.
    - It reached a test accuracy score of *40%*.
- **Model 02: GRU Stack with Batch Norm and Dense Layer**
    - Starts with Batch Normalization to stabilize the inputs.
    - Uses two GRU cells stacked together in a single RNN layer.
    - It also introduces a dropout and ultimately a dense layer with softmax to do predictions.
    - It reached a test accuracy score of *50.83%*.
- **Model 03: Two-Layer LSTM Model with Dropout**
    - Stacks two LSTM layers to deeply learn from the input sequence.
    - Adds dense layer with ReLU to refine the information further.
    - It reached a test accuracy score of *60.83%*.
- **Model 04: LSTM Stack with Batch Normalization and Learning Rate Decay**
    - Starts with Batch Normalization on the input.
    - Uses two LSTM cells inside an RNN layer, similar to Model 03.
    - Adds another Batch Normalization after the LSTM output.
    - Follows with Dense and Dropout layers.

- Uses a custom optimizer with learning rate decay, helping the model train more smoothly by reducing learning rate over time.
- It reached a test accuracy score of *66.67%*

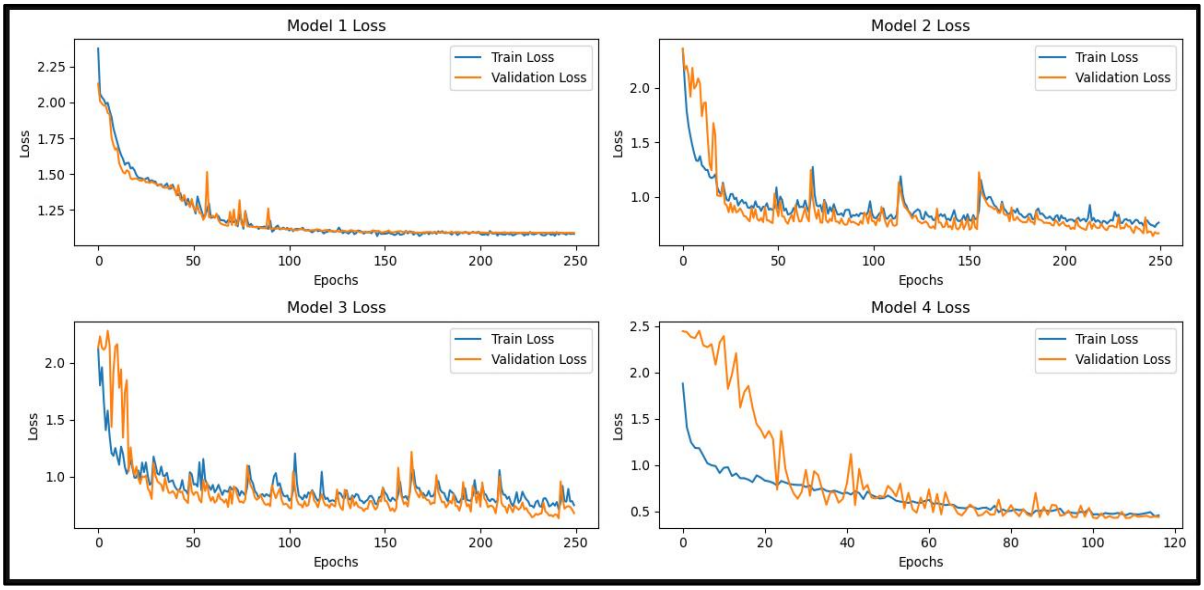


Figure 4-5: RNN Loss Curves [26]

The accuracy of the RNN models gradually improved as the architectures became more advanced and better regularized. Early models, such as simple GRU or LSTM layers showed moderate performance, as in Figure 4-5, but accuracy increased in later models due to the addition of techniques such as batch normalization, dropout, dense bottleneck layers, and learning rate scheduling. These enhancements helped stabilize training, reduce overfitting, and extract more meaningful patterns from the data. However, the accuracy still remained between 60% and 70%, and possible reasons may be the same as those discussed in the shortcomings of DenseNet performance.

4.2.4 Transformer

A Python script, or rather a data preparation pipeline, was coded to preprocess and prepare time-series (or sequence) data from multiple CSV files. Organized into categorized directories, for sequential forecasting using a transformer. The script systematically navigated through a specified directory, treating each subfolder as a distinct data category (or "label"). For every CSV file encountered, the script first loads the data, removes the 'TIME' column if present, and ensures consistent column structure across all files by reindexing and filling missing values. A crucial step involved standardizing the numerical data using StandardScaler, which transformed features to have zero mean and unit variance, making them comparable regardless of their original scales. Finally, the code segmented this processed data into fixed-length "sequences" (of 50 rows each) and associated each sequence with its corresponding category label, ultimately stacking all sequences and labels into NumPy arrays [28].

After processing all the files, it gathers the individual sequences into a single data block (all sequences) and organizes the corresponding labels into a simple list (all labels).

```
Total sequences: (9386, 50, 96)
Label map: {0: 'SLBIC', 1: 'FLB', 2: 'SP', 3: 'SGBTR', 4:
'LLB', 5: 'TT', 6: 'RI', 7: 'SGATR', 8: 'LACP', 9: 'LOCAC',
10: 'RW', 11: 'MD', 12: 'ATWS', 13: 'LOCA', 14: 'SLBOC',
15: 'LR', 16: 'Normal', 17: 'LOF'}
```

In essence, this code takes raw, messy data spread across many CSV files in different categories and cleans and standardizes it. It then organizes the data into neat, fixed-size chunks, each with its own category label. In Figure 4-6, a data preparation pipeline for Transformer feeding is shown [29].

```
all_sequences = []
all_labels = []
label_map = {}
label_idx = 0
reference_columns = None

for folder in os.listdir(data_dir):
    folder_path = os.path.join(data_dir, folder)
    if os.path.isdir(folder_path) and not folder.startswith('.'):
        label_map[label_idx] = folder
        for file in os.listdir(folder_path):
            file_path = os.path.join(folder_path, file)
            if not file.lower().endswith('.csv'):
                continue
            try:
                df = pd.read_csv(file_path)
                if reference_columns is None:
                    reference_columns = df.columns.tolist()
                    if 'TIME' in reference_columns:
                        reference_columns.remove('TIME')
                df = df.drop(columns=['TIME'], errors='ignore')

                # Align columns
                df = df.reindex(columns=reference_columns)
                df = df.fillna(0)

                scaler = StandardScaler()
                df_scaled = scaler.fit_transform(df)

                for i in range(0, len(df_scaled) - sequence_length + 1, sequence_length):
                    seq = df_scaled[i:i + sequence_length]
                    all_sequences.append(seq)
                    all_labels.append(label_idx)
            except Exception as e:
                print(f"Failed to process {file_path}: {e}")
        label_idx += 1

all_sequences = np.stack(all_sequences)
all_labels = np.array(all_labels)

print(" Total sequences:", all_sequences.shape)
print(" Label map:", label_map)
```

Figure 4-6: Data Preparation for Transformer [25]

4.2.4.1 03-Encoder Block Transformer

```python
#----------------------------
# DEFINING MODEL ARCHITECTURE
#----------------------------

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split

class TransformerModel(nn.Module):
    def __init__(self, input_size, num_classes, num_heads=4, num_layers=3, hidden_dim=64, dropout=0.1):
        super(TransformerModel, self).__init__()

        self.encoder_layer = nn.TransformerEncoderLayer(d_model=input_size, nhead=num_heads, dim_feedforward=hidden_dim, dropout=dropout)
        self.transformer_encoder = nn.TransformerEncoder(self.encoder_layer, num_layers=num_layers)
        self.fc = nn.Linear(input_size, num_classes)

    def forward(self, x):
        x = self.transformer_encoder(x)
        x = x.mean(dim=1)
        x = self.fc(x)
        return x

# Model parameters
input_size = 96
num_classes = 18

# Initialize the model
model = TransformerModel(input_size=input_size, num_classes=num_classes)
print(model)
```

Figure 4-7: 03-Encoder Block Transformer [25]

The Transformer model was built from the following important parts as depicted in Figure 4-7:

1. The Building Block: (TransformerEncoderLayer)
    - self_attn (Multi-head Attention)
    - linear1 & linear2 (FeedForward Network)
    - LayerNorm
    - Dropuout
2. Stacking the Blocks: (transformer_encoder)
    - Instead of just one TransformerEncoderLayer, this model uses three of them, stacked on top of the other. This is indicated by 3 x TransformerEncoderLayer.
    - Stacking makes the model learn about patterns and relations in the data sequence.
3. The Final Step: fc (Fully Connected Layer)
    - After all the data has been processed by the transformer encoder layers, this final layer takes this information and transforms it into the output.

- The out_features=18 suggests that this model is designed to classify input sequences into 18 different categories.

Then the processed data (all_sequences and all_labels) was divided into training and testing datasets, 80 percent of the data were aimed to learn and 20 percent of the data were reserved to test. Data was then converted into PyTorch tensors and loaded into DataLoaders, which efficiently feed batches of data to the model. A loss function (CrossEntropyLoss) was defined to measure the model's prediction errors and an optimizer (Adam) to adjust the model's internal parameters to reduce those errors. Finally, a training loop was initiated that ran for 20 epochs as shown in Figure 4-8.

This 03-Encoder Block Transformer hit 94.25% accuracy.

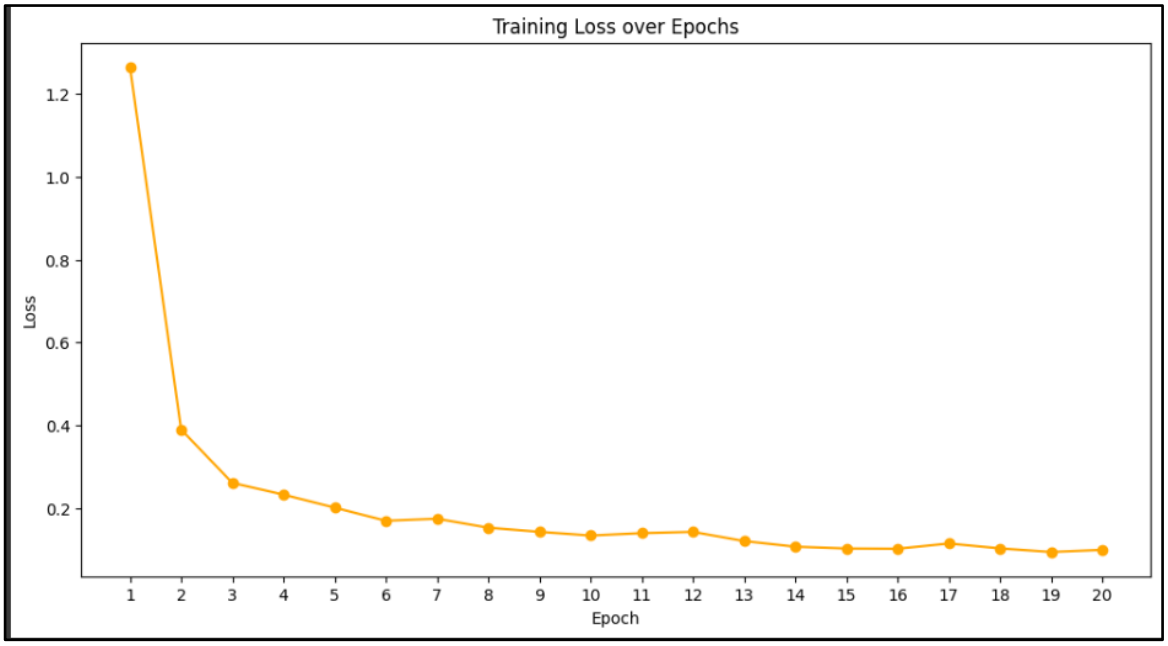


Figure 5: Transformer Loss Curve [25]

4.2.4.2 Continuous Monitoring with the Transformer

After training and testing the Transformer model and saving it, new python script was coded to monitor a folder in real time, looking for new .csv files containing sensor readings and then detect if any abnormal event is happening based on those readings using a pre-trained Transformer model. Now what the script actually does is [25]:

- Loads the saved model (i.e., trained transformer) from a .pth file.
- It also uses a saved (trained on the dataset) StandardScaler to normalize the input data, ensuing values are within a consistent range as it was during the training.
- It keeps watching a specified folder (we can also change the path) /live_data for new .csv files.
- Every few seconds (default is set to 5), it checks if new files have arrived and do the required CSV file processing.
- Now for each sequence, the model predicts a class label (e.g., “LOCA”, “NORMAL”, “SGATR”, etc.).
- If the label is anything other than “NORMAL”, it reports the predicted anomaly name.
- If it’s “NORMAL”, it simply logs that as safe information.

This setup is for streaming data or schedule dumps from systems. It can be used to automate safety checks, detect faults early, or monitor ongoing operations continuously.

Conclusion

This research work was aimed at creating an efficient anomaly detection system in nuclear power plants (NPP) and this is of great concern to identify the deviations in the operation process in a timely manner to guarantee its safety and reliability. To characterize the behavior of different machine learning and deep learning models, the Nuclear Power Plant Accident Dataset (NPPAD) based on PCTRAN models using a two-loop Pressurized Water Reactor (PWR) simulations was used. The behavior of the different models to analyze the attributes of the dataset that will be applied in modeling, understanding the circumstances during a nuclear accident and forecasting the abnormalities with regard to time-dependent data.

The results of all models tested showed the relevance of architecture decisions and training approaches in classification and prediction results. Compared to the other classification models, the single-layer MLP (Model 01) showed a high-test accuracy of 92.50% which depicts the high performance of the model in classification of the type of accidents in the nuclear plant. A more complicated structure of MLP, such as the four-layer and the two-layer models, exhibited less accuracy and stability necessitating early stopping. The CNN demonstrated a fair accuracy of 75.83%, reasonably treating the reshaped data of the input, and the DenseNet architecture was quite low in terms of test performance, 54 to 56%, potentially because of a possible over-fitting situation or a similarity issue within the model and information in the dataset. In sequential anomaly forecasting, LSTM-based models mostly got better results compared to time series, with the LSTM model with batch normalization and learning rate decay (Model 04) achieving 66.67 percent accuracy. But among all the models, the Transformer model having encoder blocks of three reached 94.25 % test accuracy.

References

[1] R. L. H. &. L. H. Mihalcea, "NLP (natural language processing) for NLP (natural language programming)," pp. 319-330, 2006.

[2] R. Patil and V. Gudivada, "A review of current trends, techniques, and challenges in large language models (LLMs)," *Applied Sciences,* vol. 14, no. 5, p. 2074, 2024.

[3] J. R. Lamarsh, Introduction to Nuclear Engineering, Addison-Wesley, 1975.

[4] J. K. Shultis, D. S. McGregor and R. E. Faw, Fundamentals of Nuclear Science and Engineering, CRC Press, 2016.

[5] S. Kim and F. Zhang, "A Robust Anomaly Detection System for Nuclear Power Plants Under Varying Environmental Conditions and Malfunction Levels," *Nuclear Technology,* pp. 1-20, 2025.

[6] X. Xiao, B. Qi, J. Liang, J. Tong, Q. Deng and P. Chen, "Enhancing LOCA Breach Size Diagnosis with Fundamental Deep Learning Models and Optimized Dataset Construction," *Energies,* vol. 17, no. 1, 2023.

[7] R. L. Murray and K. E. Holbert, Nuclear Energy: An Introduction to the Concepts, Systems, and Applications of Nuclear Processes, Elsevier, 2014.

[8] B. Qi, X. Xiao, J. Liang, L. C. C. Po, L. Zhang and J. Tong, "An Open Time-Series Simulated Dataset Covering Various Accidents for Nuclear Power Plants," *Scientific Data,* vol. 9, no. 1, 2022.

[9] T. Hamidouche, A. Bousbia-Salah and F. D'Auria, "Overview of Accident Analysis in Nuclear Research Reactors," *Progress in Nuclear Energy,* vol. 50, no. 1, pp. 7-14, 2008.

[10] A. Chaudhary, J. Han, S. Kim, A. Kim and S. Choi, "Anomaly Detection and Analysis in Nuclear Power Plants," *Electronics,* vol. 13, no. 22, 2024.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, ... and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu and L. Shao, "Normalization Techniques in Training DNNs: Methodology, Analysis and Application," *IEEE Transactions on Pattern Analysis and Machine Intelligence,* vol. 45, no. 8, p. 10173–10196, 2023.

[13] A. Zeyer, P. Bahar, K. Irie, R. Schlüter and H. Ney, "A Comparison of Transformer and LSTM Encoder Decoder Models for ASR," in *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2019.

[14] CampusX, "100 Days of Deep Learning," [Online]. Available: https://youtube.com/playlist?list=PLKnIA16_RmvYuZauWaPlRTC54KxSNLtNn&si=K26n2dpfnS1w5NT8.

[15] A. D. Dongare, R. R. Kharde and A. D. Kachare, "Introduction to Artificial Neural Network," *International Journal of Engineering and Innovative Technology (IJEIT),* vol. 2, no. 1, pp. 189-194, 2012.

[16] S. Sharma, S. Sharma and A. Athaiya, "Activation Functions in Neural Networks," *Towards Data Science,* vol. 6, no. 12, p. 310–316, 2017.

[17] A. Karpathy, "Yes you should understand backprop," 20 December 2016. [Online]. Available: https://karpathy.medium.com/yes-you-should-understand-backprop-e2f06eab496b.

[18] L. O. Chua, "CNN: A Vision of Complexity," *International Journal of Bifurcation and Chaos,* vol. 7, no. 10, p. 2219–2425, 1997.

[19] MathWorks, "What Is a Convolutional Neural Network?," [Online]. Available: https://www.mathworks.com/discovery/convolutional-neural-network.html.

[20] D. Bhatt, C. Patel, H. Talsania, J. Patel, R. Vaghela, S. Pandya, ... and H. Ghayvat, "CNN Variants for Computer Vision: History, Architecture, Application, Challenges and Future Scope," *Electronics,* vol. 10, no. 20, 2021.

[21] p. gudikandula, "A Beginner Intro to Convolutional Neural Networks," 23 March 2019. [Online]. Available: https://purnasaigudikandula.medium.com/a-beginner-intro-to-convolutional-neural-networks-684c5620c2ce.

[22] A. Sherstinsky, "Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network," *Physica D: Nonlinear Phenomena,* vol. 404, 2020.

[23] T. Katte, "Recurrent Neural Network and Its Various Architecture Types," *International Journal of Research and Scientific Innovation,* vol. 5, no. 3, p. 124–129, 2018.

[24] B. Qi, Y. Wang, X. Xiao and J. Liang, "Underlying Deep Learning Networks Diagnosis Evaluation and Generative Adversarial Network Data Augmentation Based on a Benchmark Accident Dataset," in *International Conference on Nuclear Engineering (ICONE)*, 2024.

[25] Admin.. [Online]. Available: https://colab.research.google.com/drive/1oFhA62HBbNCYcPaVACnpXf3kewtud_g4?usp=sharing.

[26] Admin.. [Online]. Available: https://drive.google.com/drive/folders/1KkLwm1YEdo8zUuavMGWhISLVXSSmiUvz?usp=sharing.

[27] J. Hou, K. Ni and A. Hawari, "An Artificial Neural Network-Based Anomaly Detection Algorithm for Nuclear Power Plants," *Transactions,* vol. 120, no. 1, pp. 219-222, 2019.

[28] F. Dong, S. Chen, K. Demachi, M. Yoshikawa, A. Seki and S. Takaya, " Attention-Based Time Series Analysis for Data-Driven Anomaly Detection in Nuclear Power Plants," *Nuclear Engineering and Design,* vol. 404, 2023.

[29] S. Yi, S. Zheng, S. Yang, G. Zhou and J. He, "Robust Transformer-Based Anomaly Detection for Nuclear Power Data Using Maximum Correntropy Criterion," *Nuclear Engineering and Technology,* vol. 56, no. 4, p. 1284–1295, 2024.

[30] B. R. U. Thomas W. Kerlin, "Pressurized water reactors," in *Dynamics and Control of Nuclear Reactors*, Academic Press, 2019, 2019.

[31] K. Erdem, "Understanding Positional Encoding in Transformers," 10 May 2021. [Online]. Available: https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers.

[32] Sachinsoni, "Cross Attention in Transformer," 6 September 2024. [Online]. Available: https://medium.com/@sachinsoni600517/cross-attention-in-transformer-f37ce7129d78.

[33] databricks, "Neural Network," [Online]. Available: https://www.databricks.com/glossary/neural-network.

[34] "CS231n Deep Learning For Computer Vision," [Online]. Available: https://cs231n.github.io/neural-networks-1/.

[35] V. Choubey, "Understanding Recurrent Neural Network (RNN) and Long Short Term Memory(LSTM)," 23 July 2020. [Online]. Available: https://medium.com/analytics-vidhya/undestanding-recurrent-neural-network-rnn-and-long-short-term-memory-lstm-30bc1221e80d.

Vita

Muhammad Daniyal Nazeer is an undergraduate in physics at The Pakistan Institute of Engineering and Applied Sciences (PIEAS), where he got aware of machine learning and data analysis, remodeling them to solve physics problems. He works smoothly on packages such as TensorFlow, PyTorch or OpenCV with deep learning algorithms such as CNN, YOLO, UNET or Transformers.

His latest work is a bachelor dissertation about the anomaly detection on the nuclear power plants using Transformer models with a priori accuracy of more than 94%, based on artificial simulated reactor incident data. In the example of detecting vehicles, Daniyal has also trained a personalized YOLOv8 model and transferred the discovered characteristics of the pre-trained models ResNet50 and VGG-16 to the process of classification. His previous internships experiences at National Centre of Physics (NCP) and Inter-Services Public Relations (ISPR) have sensitized him with the minimal assumptions of AI, Quantum Computing, and career building.

As a Womanium Global Quantum Scholarship recipient, Daniyal has studied quantum gates and circuits and coded them with Qiskit. He also possesses some certifications in machine learning and data analytics like DeepLearning.ai, DataCamp and OpenCV University. Irrespective of these scholarly pursuits, he worked with the community in the field of physics and AI.